Animixel

0.0.1

Generated by Doxygen 1.18.0

1 Data Structure Index 1

1.1 Data Structures . . . 1

2 File Index 3

2.1 File List . . . 3

3 Data Structure Documentation 5

3.1 Canvas Struct Reference . . . 5

3.1.1 Detailed Description . . . 5

3.2 Input_Handler Struct Reference . . . 5

3.2.1 Detailed Description . . . 6

4 File Documentation 7

4.1 src/canvas.h File Reference . . . 7

4.1.1 Detailed Description . . . 8

4.1.2 Function Documentation . . . 8

4.1.2.1 canvas_bresenham() . . . 8

4.1.2.2 canvas_clear_pixel() . . . 8

4.1.2.3 canvas_draw() . . . 9

4.1.2.4 canvas_free() . . . 9

4.1.2.5 canvas_init() . . . 9

4.1.2.6 canvas_print() . . . 10

4.1.2.7 canvas_set_pixel() . . . 10

4.2 canvas.h . . . 10

4.3 src/input.h File Reference . . . 11

4.3.1 Detailed Description . . . 12

4.3.2 Enumeration Type Documentation . . . 12

4.3.2.1 keystate . . . 12

4.3.3 Function Documentation . . . 12

4.3.3.1 cursor_in_bounds() . . . 12

4.3.3.2 glfw_key_callback() . . . 13

4.3.3.3 glfw_mouse_callback() . . . 13

4.3.3.4 input_handler_init() . . . 14

4.3.3.5 update_key_state() . . . 14

4.4 input.h . . . 15

Index 17

Chapter 1

Data Structure Index

1.1 Data Structures

Here are the data structures with brief descriptions:

Canvas
: Struct that describes a drawing canvas 5

Input_Handler
: Struct that acts as input handling interface 5

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

src/canvas.h
Contains the definition for a Canvas and its associated functions 7
src/input.h
Contains structs/functions for handling user input 11

Chapter 3

Data Structure Documentation

3.1 Canvas Struct Reference

Struct that describes a drawing canvas.

`#include <canvas.h>`

Data Fields

- uint32_t * **pixels**
  Array holding the pixel data.
- BOB_Pixelbuffer_Handle **pb**
  Reference handle to the pixelbuffer that actually renders the canvas data to the screen.
- size_t **width**
  Width of the canvas.
- size_t **height**
  Height of the canvas.
- size_t **scale**
  Scale factor of the canvas. Used when rendering.

3.1.1 Detailed Description

Struct that describes a drawing canvas.

The documentation for this struct was generated from the following file:

- src/canvas.h

3.2 Input_Handler Struct Reference

Struct that acts as input handling interface.

`#include <input.h>`

Data Fields

- keystate **key_status** [NUM_KEYS]

  Map that holds the state of all the current keys we care about.
- double **mouse_x**

  Mouse x position in the current frame.
- double **mouse_y**

  Mouse y position in the current frame.
- double **prev_x**

  Mouse x position in the previous frame.
- double **prev_y**

  Mouse y position in the previous frame.

3.2.1 Detailed Description

Struct that acts as input handling interface.

The documentation for this struct was generated from the following file:

- src/input.h

Chapter 4

File Documentation

4.1 src/canvas.h File Reference

Contains the definition for a Canvas and its associated functions.

```
#include <stdint.h>
#include <stddef.h>
#include <bob.h>
```

Data Structures

- struct Canvas

  Struct that describes a drawing canvas.

Functions

- void canvas_init (Canvas *c, size_t width, size_t height, size_t scale, uint32_t default_colour, BOB_↩ Renderer_Handle r)

  Initialises a Canvas.
- void canvas_free (Canvas *c, BOB_Renderer_Handle r)

  Frees all memory used by member variables of a Canvas outside of the struct itself, but does not destroy the memory region occupied by the struct.
- void canvas_draw (Canvas *c, BOB_Renderer_Handle renderer)

  Draws a canvas to the screen.
- void canvas_set_pixel (Canvas *c, uint32_t x, uint32_t y, uint32_t colour)

  Sets a pixel in the Canvas.
- void canvas_clear_pixel (Canvas *c, uint32_t x, uint32_t y)

  Erases a pixel in the Canvas by setting its RGBA colour to (0, 0, 0, 0).
- void canvas_print (Canvas *c)

  Prints the pixels array of the Canvas to the terminal.
- void canvas_bresenham (Canvas *c, uint32_t sx, uint32_t sy, uint32_t ex, uint32_t ey, uint32_t colour)

  Uses Bresenham's line algorithm adapted from here: https://www.geeksforgeeks.org/dsa/↩ bresenhams-line-generation-algorithm/ to draw a line between two points on a Canvas.

4.1.1 Detailed Description

Contains the definition for a Canvas and its associated functions.

4.1.2 Function Documentation

4.1.2.1 canvas_bresenham()

```
void canvas_bresenham (
            Canvas * c,
            uint32_t sx,
            uint32_t sy,
            uint32_t ex,
            uint32_t ey,
            uint32_t colour)
```

Uses Bresenham's line algorithm adapted from here: https://www.geeksforgeeks.org/dsa/bresenhams-line-generation-algorithm/ to draw a line between two points on a Canvas.

Parameters

c	a pointer to the Canvas where the line should be drawn
sx	the starting x coord of the line
sy	the starting y coord of the line
ex	the ending x coord of the line
ey	the ending y coord of the line
colour	the colour the line whould be drawn in

4.1.2.2 canvas_clear_pixel()

```
void canvas_clear_pixel (
            Canvas * c,
            uint32_t x,
            uint32_t y)
```

Erases a pixel in the Canvas by setting its RGBA colour to (0, 0, 0, 0).

Parameters

c	a pointer to the Canvas where the pixel should be erased
x	the x coordinate of the pixel
y	the y coordinate of the pixel

4.1.2.3 canvas_draw()

```
void canvas_draw (
            Canvas * c,
            BOB_Renderer_Handle renderer)
```

Draws a canvas to the screen.

Updates the Canvas' pixelbuffer with the new pixel values from the pixels array. Determines the size of the destination quad using the width, height and scale member variables

Parameters

c	a pointer to the Canvas to be drawn
r	a handle to the renderer that was used to create the Canvas' pixelbuffer

4.1.2.4 canvas_free()

```
void canvas_free (
            Canvas * c,
            BOB_Renderer_Handle r)
```

Frees all memory used by member variables of a Canvas outside of the struct itself, but does not destroy the memory region occupied by the struct.

Parameters

c	a pointer to the Canvas whole members must be freed
r	a handle to the renderer that was used to create the Canvas' pixelbuffer

4.1.2.5 canvas_init()

```
void canvas_init (
            Canvas * c,
            size_t width,
            size_t height,
            size_t scale,
            uint32_t default_colour,
            BOB_Renderer_Handle r)
```

Initialises a Canvas.

Does not allocate memory for the struct itself (this must be done beforehand using either a local struct variable or a heap allocation), but will allocate memory for the pixels buffer and will create the PixelBuffer via a call to BOB

Parameters

c	a pointer to a Canvas to be initialised
width	the desired width of the Canvas

height	the desired height of the Canvas
scale	the desired scale factor of the Canvas
default_colour	the background colour the Canvas should default to when first drawn
r	the handle of the BOB_Renderer the Canvas should use to generate its pixelbuffer

4.1.2.6 canvas_print()

```
void canvas_print (
            Canvas * c)
```

Prints the pixels array of the Canvas to the terminal.

Used for debugging and should not be used in production code

Parameters

c	a pointer to the Canvas whose pixels should be printed

4.1.2.7 canvas_set_pixel()

```
void canvas_set_pixel (
            Canvas * c,
            uint32_t x,
            uint32_t y,
            uint32_t colour)
```

Sets a pixel in the Canvas.

Only updates the pixels array, not the pixelbuffer

Parameters

c	a pointer to the Canvas where the pixel should be set
x	the x coordinate of the pixel
y	the y coordinate of the pixel
colour	the new colour the pixel should be set to

4.2 canvas.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef CANVAS_H
00007 #define CANVAS_H
00008
00009 #include <stdint.h>
00010 #include <stddef.h>
00011 #include <bob.h>
00012
00014 typedef struct {
```

```
00015     uint32_t *pixels;
00016     BOB_Pixelbuffer_Handle pb;
00017     size_t width;
00018     size_t height;
00019     size_t scale;
00020 } Canvas;
00021
00033 void canvas_init(Canvas *c, size_t width, size_t height, size_t scale, uint32_t default_colour,
      BOB_Renderer_Handle r);
00040 void canvas_free(Canvas *c, BOB_Renderer_Handle r);
00047 void canvas_draw(Canvas *c, BOB_Renderer_Handle renderer);
00055 void canvas_set_pixel(Canvas *c, uint32_t x, uint32_t y, uint32_t colour);
00062 void canvas_clear_pixel(Canvas *c, uint32_t x, uint32_t y);
00068 void canvas_print(Canvas *c);
00079 void canvas_bresenham(Canvas *c, uint32_t sx, uint32_t sy, uint32_t ex, uint32_t ey, uint32_t colour);
00080
00081 #endif
```

4.3 src/input.h File Reference

Contains structs/functions for handling user input.

```
#include <GLFW/glfw3.h>
```

Data Structures

- struct Input_Handler

  Struct that acts as input handling interface.

Enumerations

- enum keystate { KEY_JUST_PRESSED , KEY_PRESSED , KEY_RELEASED , KEY_JUST_RELEASED }

  Enum to hold the state of the keys.
- enum keys {
  KEY_0 , **KEY_1** , **KEY_2** , **KEY_3** ,
  KEY_4 , **KEY_5** , **KEY_6** , **KEY_7** ,
  KEY_8 , **KEY_9** , **KEY_A** , **KEY_B** ,
  KEY_C , **KEY_D** , **KEY_E** , **KEY_F** ,
  KEY_G , **KEY_H** , **KEY_I** , **KEY_J** ,
  KEY_K , **KEY_L** , **KEY_M** , **KEY_N** ,
  KEY_O , **KEY_P** , **KEY_Q** , **KEY_R** ,
  KEY_S , **KEY_T** , **KEY_U** , **KEY_V** ,
  KEY_W , **KEY_X** , **KEY_Y** , **KEY_Z** ,
  KEY_MINUS , **KEY_PLUS** , **MOUSE_BUTTON_LEFT** , **MOUSE_BUTTON_RIGHT** ,
  MOUSE_BUTTON_MIDDLE , **NUM_KEYS** }

  Enum that holds all of the keys we currently want to work with.

Functions

- void input_handler_init (Input_Handler *ih)

  Initialises an Input_Handler.
- void glfw_mouse_callback (GLFWwindow *window, int key, int action, int mods)

  Callback function to be used by GLFW when a mouse button's state changes.
- void glfw_key_callback (GLFWwindow *window, int key, int scancode, int action, int mods)

  Callback function to be used by glfw when a key's state changes.

- void **glfw_cursor_pos_callback** (GLFWwindow *window, double xpos, double ypos)

  Callback function to be used by glfw when the cursor position changes.
- void update_key_state (Input_Handler *ih)

  Updates the key state after user input processing.
- int cursor_in_bounds (Input_Handler *ih, float min_x, float min_y, float max_x, float max_y)

  Simple checker that returns if the cursor is in the bounds [min_x, max_x), [min_y, max_y).

Variables

- Input_Handler **handler**

  Global for use in the callback functions.

4.3.1 Detailed Description

Contains structs/functions for handling user input.

4.3.2 Enumeration Type Documentation

4.3.2.1 keystate

```
enum keystate
```

Enum to hold the state of the keys.

Enumerator

KEY_JUST_PRESSED	Was pressed this frame.
KEY_PRESSED	Held on from the previous frame.
KEY_RELEASED	Not pressed.
KEY_JUST_RELEASED	Released the previous frame.

4.3.3 Function Documentation

4.3.3.1 cursor_in_bounds()

```
int cursor_in_bounds (
            Input_Handler * ih,
            float min_x,
            float min_y,
            float max_x,
            float max_y)
```

Simple checker that returns if the cursor is in the bounds [min_x, max_x), [min_y, max_y).

Parameters

ih	a pointer to the Input_Handler whose mouse position we are checking
min_x	the minimum x value the cursor can hold
min_y	the minimum y value the cursor can hold
max_x	the maximum x value the cursor can hold
max_y	the maximum y value the cursor can hold

Returns

1 if the cursor is in the bounds of the window, 0 otherwise

4.3.3.2 glfw_key_callback()

```
void glfw_key_callback (
            GLFWwindow * window,
            int key,
            int scancode,
            int action,
            int mods)
```

Callback function to be used by glfw when a key's state changes.

If the new state is GLFW_PRESS and the old state stored in the input_handler was KEY_JUST_RELEASED or KEY_RELEASED, sets the state to KEY_JUST_PRESSED. If the new state is GLFW_PRESS and the old state stored in the input_handler was KEY_PRESSED or KEY_JUST_PRESSED, sets the state to KEY_PRESSED. If the new state is GLFW_RELEASE and the old state stored in the input_handler was KEY_JUST_RELEASED or KEY_RELEASED, sets the state to KEY_RELEASED. If the new state is GLFW_RELEASE and the old state stored in the input_handler was KEY_JUST_PRESSED or KEY_PRESSED, sets the state to KEY_JUST_RELEASED. If the new state is GLFW_REPEAT set the state to KEY_PRESSED.

Parameters

window	a pointer to the GLFWwindow where the key change occured
key	the key whose state has changed
scancode	the hardware specific scancode of the key. Not used by this callback implementation
action	the new state of the key
mods	modifier bits. Not used by this callback implementation

4.3.3.3 glfw_mouse_callback()

```
void glfw_mouse_callback (
            GLFWwindow * window,
            int key,
```

```
            int action,
            int mods)
```

Callback function to be used by GLFW when a mouse button's state changes.

If the new state is GLFW_PRESS and the old state stored in the input_handler was KEY_JUST_RELEASED or KEY_RELEASED, sets the state to KEY_JUST_PRESSED. If the new state is GLFW_PRESS and the old state stored in the input_handler was KEY_PRESSED or KEY_JUST_PRESSED, sets the state to KEY_PRESSED. If the new state is GLFW_RELEASE and the old state stored in the input_handler was KEY_JUST_RELEASED or KEY_RELEASED, sets the state to KEY_RELEASED. If the new state is GLFW_RELEASE and the old state stored in the input_handler was KEY_JUST_PRESSED or KEY_PRESSED, sets the state to KEY_JUST_RELEASED.

Parameters

window	a pointer to the GLFWwindow where the mouse button change occured
key	the mouse button whose state has changed
action	the new state of the mouse button
mods	modifier bits. Not used by this callback implementation

4.3.3.4 input_handler_init()

```
void input_handler_init (
            Input_Handler * ih)
```

Initialises an Input_Handler.

Sets the initial cursor position to (0, 0) and all of the keys to KEY_RELEASED Does not allocate memory for the Input_Handler, this must be done before the function was called

Parameters

ih	a pointer to the Input_Handler to be initialised

4.3.3.5 update_key_state()

```
void update_key_state (
            Input_Handler * ih)
```

Updates the key state after user input processing.

Parameters

ih	a pointer to the Input_Handler whose key map needs updating

4.4 input.h

Go to the documentation of this file.

```
00001
00005
00006 #ifndef __INPUT_H__
00007 #define __INPUT_H__
00008 #include <GLFW/glfw3.h>
00009
00011 typedef enum {
00012     KEY_JUST_PRESSED,
00013     KEY_PRESSED,
00014     KEY_RELEASED,
00015     KEY_JUST_RELEASED
00016 } keystate;
00017
00019 typedef enum {
00020     KEY_0,
00021     KEY_1,
00022     KEY_2,
00023     KEY_3,
00024     KEY_4,
00025     KEY_5,
00026     KEY_6,
00027     KEY_7,
00028     KEY_8,
00029     KEY_9,
00030     KEY_A,
00031     KEY_B,
00032     KEY_C,
00033     KEY_D,
00034     KEY_E,
00035     KEY_F,
00036     KEY_G,
00037     KEY_H,
00038     KEY_I,
00039     KEY_J,
00040     KEY_K,
00041     KEY_L,
00042     KEY_M,
00043     KEY_N,
00044     KEY_O,
00045     KEY_P,
00046     KEY_Q,
00047     KEY_R,
00048     KEY_S,
00049     KEY_T,
00050     KEY_U,
00051     KEY_V,
00052     KEY_W,
00053     KEY_X,
00054     KEY_Y,
00055     KEY_Z,
00056     KEY_MINUS,
00057     KEY_PLUS,
00058     MOUSE_BUTTON_LEFT,
00059     MOUSE_BUTTON_RIGHT,
00060     MOUSE_BUTTON_MIDDLE,
00061     NUM_KEYS
00062 } keys;
00063
00065 typedef struct {
00066     keystate key_status[NUM_KEYS];
00067     double mouse_x;
00068     double mouse_y;
00069     double prev_x;
00070     double prev_y;
00071 } Input_Handler;
00072
00073 extern Input_Handler handler;
00074
00081 void input_handler_init(Input_Handler *ih);
00097 void glfw_mouse_callback(GLFWwindow *window, int key, int action, int mods);
00115 void glfw_key_callback(GLFWwindow *window, int key, int scancode, int action, int mods);
00119 void glfw_cursor_pos_callback(GLFWwindow *window, double xpos, double ypos);
00124 void update_key_state(Input_Handler *ih);
00134 int cursor_in_bounds(Input_Handler *ih, float min_x, float min_y, float max_x, float max_y);
00135
00136 #endif
```

Index

Canvas, 5
canvas.h
    canvas_bresenham, 8
    canvas_clear_pixel, 8
    canvas_draw, 8
    canvas_free, 9
    canvas_init, 9
    canvas_print, 10
    canvas_set_pixel, 10
canvas_bresenham
    canvas.h, 8
canvas_clear_pixel
    canvas.h, 8
canvas_draw
    canvas.h, 8
canvas_free
    canvas.h, 9
canvas_init
    canvas.h, 9
canvas_print
    canvas.h, 10
canvas_set_pixel
    canvas.h, 10
cursor_in_bounds
    input.h, 12

glfw_key_callback
    input.h, 13
glfw_mouse_callback
    input.h, 13

input.h
    cursor_in_bounds, 12
    glfw_key_callback, 13
    glfw_mouse_callback, 13
    input_handler_init, 14
    KEY_JUST_PRESSED, 12
    KEY_JUST_RELEASED, 12
    KEY_PRESSED, 12
    KEY_RELEASED, 12
    keystate, 12
    update_key_state, 14
Input_Handler, 5
input_handler_init
    input.h, 14

KEY_JUST_PRESSED
    input.h, 12
KEY_JUST_RELEASED
    input.h, 12
KEY_PRESSED
    input.h, 12
KEY_RELEASED
    input.h, 12
keystate
    input.h, 12

src/canvas.h, 7, 10
src/input.h, 11, 15

update_key_state
    input.h, 14